

This is actually one of the biggest architecture questions. **CockroachDB** is excellent for transactional data and persistent agent memory, but it is not meant to become a permanent archive of every news article ever published. The solution is to use **tiered storage**.

What are the practical limitations?

For a hackathon, you may start with one month of data, but in production imagine:

```
500 articles/hour  
  
≈ 12,000/day  
  
≈ 360,000/month  
  
≈ 4.3 million/year
```

Each article stores:

- Full text
- Embedding (1536 dimensions)
- Metadata
- Bias score
- Sentiment

Storage grows rapidly, and vector indexes become larger and slower.

CockroachDB should not store everything

Think of CockroachDB as **active memory**, similar to how humans remember recent and important events, not every newspaper they've ever read.

Separate data into three layers.

```
Hot Data  
↓  
  
Warm Data  
↓  
  
Cold Archive
```

Tier 1 — Hot Memory (CockroachDB)

This is the agent's working memory.

Contains:

- Recent articles (last 7–30 days)

- User profiles
- Reading history
- Agent memories
- Active embeddings
- Vector index

CockroachDB

✓ Fast vector search

✓ Fast SQL

✓ Agent memory

✓ Recent news

This is where almost every user query is answered.

Tier 2 — Cold Storage (Amazon S3)

After articles become old:

30 days old

↓

Move to S3

Store

JSON

HTML

Markdown

Parquet

Compressed text

S3 is extremely cheap.

Instead of paying database storage costs,

pay object storage costs.

Tier 3 — Metadata Only

Instead of deleting old articles entirely,
keep lightweight metadata inside CockroachDB.

Example

```
id  
  
title  
  
topic  
  
source  
  
date  
  
S3_path
```

Instead of

```
id  
  
title  
  
FULL ARTICLE  
  
embedding  
  
metadata  
  
everything...
```

When needed

```
User  
  
↓  
  
Requests old article  
  
↓  
  
CockroachDB returns  
  
s3://archive/...  
  
↓
```

Backend fetches from S3

Archive Strategy

Use lifecycle policies.

Example

0-30 days

CockroachDB

↓

30-180 days

S3 Standard

↓

180+ days

S3 Glacier

AWS automatically moves files.

No Lambda required.

Embedding Retention Strategy

Do you really need vectors forever?

Probably not.

Keep vectors only for recent news.

Recent

✓ Embedding

✓ Vector Index

✓ Similarity Search

Old

Archive only

No vector

No index

If someone requests an archived article,
you can regenerate the embedding on demand.

User Memory Never Expires

Notice the distinction:

Articles become stale.

User memory does not.

Keep permanently:

User profile

Reading history

Interest vectors

Political leaning

Feedback

Agent memories

These are tiny compared to article storage.

Global Deduplication

The same Reuters story appears on

- NewsAPI
- Guardian
- GNews
- SerpAPI

Don't store duplicates.

Create a content hash.

SHA256(

Title

+

Published Time

+

Source

)

If hash exists

Ignore article

You'll save a surprising amount of storage.

Top-K Retention

Not every article deserves long-term storage.

Keep only

High credibility

OR

Trending

OR

Read by users

Discard

Local spam

Low-quality blogs

Duplicates

Unread articles

Sliding Window Vector Index

Instead of indexing every article,

only index

Last 30 days

When article expires

Delete vector

Move article to S3

Your vector index stays small and fast.

A Better Architecture



One more optimization: Don't embed everything

Your current design embeds every fetched article immediately. That's expensive and unnecessary.

A better pipeline is:

Fetch article

↓

Deduplicate

↓

Score importance

↓

Important?

Yes | No
↓

Generate embedding

↓

Store in CockroachDB

Importance can be based on:

- Trusted source (Reuters, AP, BBC)
- Trending score
- Matches at least one user's interests
- Breaking news
- High engagement

Everything else can be stored in S3 without a vector.

What this means for the hackathon

This architecture strengthens your submission because it demonstrates production thinking:

- **CockroachDB** is used as the **agent's active, persistent memory** (recent articles, user state, vectors, transactional data).
- **Amazon S3** is the **long-term archive**, not a second database.
- **Lifecycle policies** automatically move aging data, minimizing operational overhead.
- **Sliding-window vector indexing** keeps semantic search fast and storage bounded.
- **Lazy embeddings and deduplication** reduce both compute and storage costs.

This is much closer to how large-scale systems like search engines and recommendation platforms are designed, while still showcasing CockroachDB's strengths rather than forcing it to store everything.